

Core Mathematics for Machine Learning

The 4 Real Pillars

Calculus · Linear Algebra · Probability · Statistics

Every equation is presented the same way:

1. **Sign / symbol details** – what every letter and symbol means
2. **The equation itself**
3. **In words** – the equation spelled out in plain language
4. **ML use case** – where this exact equation shows up in machine learning
5. **Worked example** – a concrete, step-by-step numeric example

Contents

1	Part I — Calculus	1
1.1	Limits – The Foundation	1
1.2	Derivatives	1
1.3	Partial Derivatives	4
1.4	The Gradient	4
1.5	Jacobian and Hessian	5
1.6	Chain Rule for Multivariable Functions (Backpropagation)	6
1.7	Taylor Series	7
1.8	Integrals	7
1.9	Optimization – Where Calculus Meets ML	8
2	Part II — Linear Algebra	11
2.1	Vectors – Basic Operations	11
2.2	Matrices – Basic Operations	15
2.3	Eigenvalues, Eigenvectors, and Decompositions	17
3	Part III — Probability	20
3.1	Foundations	20
3.2	Random Variables and Distributions	23
3.3	Common Probability Distributions	25
3.4	Limit Theorems and Information Theory	28
3.5	Estimation	30
4	Part IV — Statistics	32
4.1	Descriptive Statistics – Measures of Central Tendency	32
4.2	Descriptive Statistics – Measures of Spread	33
4.3	Shape of a Distribution	35
4.4	Relationships Between Variables	36
4.5	Estimation and Confidence	38
4.6	Statistics in Model Fitting	40

1 Part I — Calculus

Calculus is the mathematics of change and slope. In ML, it is the engine behind every training loop: every time a model “learns,” calculus is computing how to nudge its parameters to reduce error.

Symbol Legend – Calculus

- $f(x)$ – a function of one variable x
- $f(\mathbf{x})$ – a function of a vector $\mathbf{x} = (x_1, \dots, x_n)$
- $f'(x)$, $\frac{df}{dx}$ – the derivative of f with respect to x
- $\partial f / \partial x_i$ – the partial derivative of f w.r.t. x_i (others held fixed)
- ∇f – the gradient (vector of all partial derivatives)
- $\nabla^2 f$, H – the Hessian matrix (matrix of second partial derivatives)
- $\lim_{x \rightarrow a}$ – “the limit as x approaches a ”
- $\int$ – integral sign (accumulation / area under a curve)
- Δx – a small change in x
- η – learning rate (a small positive step size)
- $\propto$ – “is proportional to”

1.1 Limits – The Foundation

Limit of a Function

Symbols / Details:

- $f(x)$ – the function being evaluated
- a – the point x is approaching
- L – the value $f(x)$ approaches

$$\lim_{x \rightarrow a} f(x) = L$$

In Words: As x gets closer and closer to a (from either side), $f(x)$ gets closer and closer to the value L , even if $f(x)$ is never actually evaluated exactly at a .

ML Use Case: Limits underpin the formal definition of the derivative, which is what every gradient-based optimizer (SGD, Adam, RMSProp) relies on.

Worked Example: Let $f(x) = x^2$ and $a = 2$. As $x \rightarrow 2$ (try $x = 1.9, 1.99, 2.01, 2.1$), $f(x) \rightarrow 3.61, 3.9601, 4.0401, 4.41$. The values close in on $L = 4$, so $\lim_{x \rightarrow 2} x^2 = 4$ – even though we never need x to literally equal 2.

1.2 Derivatives

Derivative – First-Principles Definition**Symbols / Details:**

- $f(x + h)$ – function value at a point shifted by a tiny amount h
- h – a tiny change in x , $h \rightarrow 0$
- $f'(x)$ – the instantaneous rate of change (slope) of f at x

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x + h) - f(x)}{h}$$

In Words: Take the change in output ($f(x + h) - f(x)$) and divide by the change in input (h), then shrink h down to nothing. What is left is the exact slope of the curve at that single point.

ML Use Case: This is the mathematical definition of “how much does the loss change if I nudge this weight a tiny bit” – the core question every training algorithm answers.

Worked Example: **Step by step:** let $f(x) = x^2$ and find $f'(3)$.

$$\begin{aligned} f'(3) &= \lim_{h \rightarrow 0} \frac{(3 + h)^2 - 3^2}{h} = \lim_{h \rightarrow 0} \frac{9 + 6h + h^2 - 9}{h} \\ &= \lim_{h \rightarrow 0} \frac{6h + h^2}{h} = \lim_{h \rightarrow 0} (6 + h) = 6 \end{aligned}$$

So the slope of $f(x) = x^2$ at $x = 3$ is exactly 6 (matches the power rule: $2x = 2(3) = 6$).

Power Rule**Symbols / Details:**

- x^n – x raised to power n
- n – any real-number exponent

$$\frac{d}{dx} [x^n] = n x^{n-1}$$

In Words: Bring the exponent down as a multiplying factor, then reduce the exponent by one.

ML Use Case: Used constantly when differentiating polynomial terms inside loss functions, e.g. the derivative of the squared-error term $(y - \hat{y})^2$.

Worked Example: $\frac{d}{dx} [x^4] = 4x^3$. Check at $x = 2$: slope = $4(2)^3 = 32$.

Sum / Difference Rule**Symbols / Details:**

- f, g – two differentiable functions

$$\frac{d}{dx} [f(x) \pm g(x)] = f'(x) \pm g'(x)$$

In Words: The derivative of a sum (or difference) is just the sum (or difference) of the individual derivatives.

ML Use Case: Lets us differentiate a loss function term-by-term, e.g. data loss + regularization term.

Worked Example: For $f(x) = x^2 + 3x$: $\frac{d}{dx}[x^2] + \frac{d}{dx}[3x] = 2x + 3$.

Product Rule

Symbols / Details:

- f, g – two differentiable functions of x

$$\frac{d}{dx}[f(x)g(x)] = f'(x)g(x) + f(x)g'(x)$$

In Words: “Derivative of the first times the second, plus the first times the derivative of the second.”

ML Use Case: Needed whenever two learned quantities are multiplied together, e.g. attention weights times values in Transformers.

Worked Example: **Step by step:** let $f(x) = x^2$, $g(x) = \sin x$.

$$\frac{d}{dx}[x^2 \sin x] = \underbrace{2x}_{f'} \cdot \sin x + x^2 \cdot \underbrace{\cos x}_{g'} = 2x \sin x + x^2 \cos x$$

Quotient Rule

Symbols / Details:

- f, g – two differentiable functions of x , $g(x) \neq 0$

$$\frac{d}{dx} \left[\frac{f(x)}{g(x)} \right] = \frac{f'(x)g(x) - f(x)g'(x)}{g(x)^2}$$

In Words: “Bottom times derivative of top, minus top times derivative of bottom, all over bottom squared.”

ML Use Case: Used to differentiate the softmax function, which is a ratio of exponentials.

Worked Example: **Step by step:** let $f(x) = x$, $g(x) = x + 1$.

$$\frac{d}{dx} \left[\frac{x}{x+1} \right] = \frac{(1)(x+1) - (x)(1)}{(x+1)^2} = \frac{1}{(x+1)^2}$$

Chain Rule (single variable)

Symbols / Details:

- $f(g(x))$ – a composite function – a function of a function
- $u = g(x)$ – the inner function, treated as an intermediate variable

$$\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x) \quad \text{or} \quad \frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

In Words: Differentiate the outer function first (leaving the inner function untouched inside it), then multiply by the derivative of the inner function.

ML Use Case: This single rule, applied repeatedly layer-by-layer, IS backpropagation – the algorithm that trains every neural network.

Worked Example: **Step by step:** let $y = (3x + 1)^2$, so outer $f(u) = u^2$, inner $u = g(x) = 3x + 1$.

$$\frac{dy}{du} = 2u = 2(3x + 1), \quad \frac{du}{dx} = 3$$

$$\frac{dy}{dx} = 2(3x + 1) \cdot 3 = 18x + 6$$

Check at $x = 1$: $y = (4)^2 = 16$; slope $= 18(1) + 6 = 24$.

1.3 Partial Derivatives

Partial Derivative

Symbols / Details:

- $f(x, y)$ – a function of two (or more) variables
- ∂ – “partial-d” symbol: differentiate w.r.t. one variable, freeze the rest

$$\frac{\partial f}{\partial x} = \lim_{h \rightarrow 0} \frac{f(x + h, y) - f(x, y)}{h}$$

In Words: Treat every variable except x as a constant, then take the ordinary derivative with respect to x only.

ML Use Case: A neural network’s loss depends on thousands/millions of weights; a partial derivative tells you the effect of changing just ONE weight while all others stay fixed.

Worked Example: **Step by step:** let $f(x, y) = x^2y + y^3$.

$$\frac{\partial f}{\partial x} = 2xy \quad (\text{treat } y \text{ as constant}), \quad \frac{\partial f}{\partial y} = x^2 + 3y^2 \quad (\text{treat } x \text{ as constant})$$

At $(x, y) = (2, 1)$: $\partial f / \partial x = 2(2)(1) = 4$, and $\partial f / \partial y = 4 + 3 = 7$.

1.4 The Gradient

Gradient Vector

Symbols / Details:

- $f(\mathbf{x})$ – a scalar-valued function of a vector $\mathbf{x} = (x_1, \dots, x_n)$
- ∇f – “nabla f” or “grad f” – the gradient, a vector of all partial derivatives

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)^\top$$

In Words: Stack every partial derivative of f into one column vector. This vector points in the direction where f increases fastest, and its length tells you how steep that increase is.

ML Use Case: The gradient of the loss function with respect to all model weights is exactly what `loss.backward()` computes in PyTorch/TensorFlow – it drives every weight update.

Worked Example: **Step by step:** let $f(x, y) = x^2 + y^2$ (a simple “bowl”).

$$\frac{\partial f}{\partial x} = 2x, \quad \frac{\partial f}{\partial y} = 2y \Rightarrow \nabla f(x, y) = (2x, 2y)^\top$$

At $(1, 2)$: $\nabla f(1, 2) = (2, 4)^\top$ – steepest uphill direction from that point.

Directional Derivative

Symbols / Details:

- $\hat{\mathbf{u}}$ – a unit vector giving a direction
- ∇f – gradient of f

$$D_{\hat{\mathbf{u}}}f(\mathbf{x}) = \nabla f(\mathbf{x}) \cdot \hat{\mathbf{u}}$$

In Words: Take the dot product of the gradient with a chosen direction vector to find the rate of change of f in exactly that direction.

ML Use Case: Explains why the negative gradient direction is the direction of steepest descent – the basis of gradient descent.

Worked Example: Using $\nabla f(1, 2) = (2, 4)^\top$ from above and direction $\hat{\mathbf{u}} = (1, 0)^\top$: $D_{\hat{\mathbf{u}}}f = (2)(1) + (4)(0) = 2$, the rate of increase purely along the x -direction.

1.5 Jacobian and Hessian

Jacobian Matrix

Symbols / Details:

- $\mathbf{f}(\mathbf{x})$ – a vector-valued function: $\mathbf{f} = (f_1, \dots, f_m)$, input $\mathbf{x} = (x_1, \dots, x_n)$
- J – the Jacobian – matrix of all first-order partial derivatives

$$J = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

In Words: Every row is one output’s gradient; every column is one input variable. Row i , column j tells you how much output i changes when input j changes.

ML Use Case: Backpropagation through an entire neural-network layer (which maps a vector of inputs to a vector of outputs) is literally multiplying Jacobians together, layer by layer.

Worked Example: Step by step: let $f_1(x, y) = x^2y$, $f_2(x, y) = x + y^2$.

$$J = \begin{bmatrix} \partial f_1 / \partial x & \partial f_1 / \partial y \\ \partial f_2 / \partial x & \partial f_2 / \partial y \end{bmatrix} = \begin{bmatrix} 2xy & x^2 \\ 1 & 2y \end{bmatrix}$$

At $(x, y) = (1, 2)$: $J = \begin{bmatrix} 4 & 1 \\ 1 & 4 \end{bmatrix}$.

Hessian Matrix

Symbols / Details:

- $f(\mathbf{x})$ – a scalar-valued function of a vector
- H – the Hessian – matrix of all second-order partial derivatives

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

In Words: Take the partial derivative of the gradient itself. Each entry tells you how the slope in one direction changes as you move in another direction – i.e. the curvature of the function.

ML Use Case: Used in second-order optimizers (Newton’s method, L-BFGS) and to check whether a critical point is a minimum, maximum, or saddle point.

Worked Example: Step by step: continue with $f(x, y) = x^2y + y^3$ from the partial-derivative example, where $f_x = 2xy$, $f_y = x^2 + 3y^2$.

$$H = \begin{bmatrix} f_{xx} & f_{xy} \\ f_{yx} & f_{yy} \end{bmatrix} = \begin{bmatrix} 2y & 2x \\ 2x & 6y \end{bmatrix}$$

At $(2, 1)$: $H = \begin{bmatrix} 2 & 4 \\ 4 & 6 \end{bmatrix}$.

1.6 Chain Rule for Multivariable Functions (Backpropagation)

Multivariable Chain Rule

Symbols / Details:

- L – final scalar loss
- z – an intermediate variable (e.g. output of one layer)
- w – a parameter feeding into z

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w}$$

In Words: To find how the final loss changes with a deep parameter, multiply the local derivatives along the path connecting that parameter to the loss.

ML Use Case: This is the exact rule applied at every node of a computational graph in reverse-mode automatic differentiation – the mechanism behind `.backward()` in every deep learning framework.

Worked Example: **Step by step:** suppose $z = wx$ (a mini linear layer) and $L = z^2$ (squared loss), with $x = 2$, $w = 3$.

$$z = wx = 6, \quad \frac{\partial L}{\partial z} = 2z = 12, \quad \frac{\partial z}{\partial w} = x = 2$$

$$\frac{\partial L}{\partial w} = 12 \times 2 = 24$$

This tells us: nudging w up by a tiny amount increases the loss at a rate of 24 – so gradient descent will push w down.

1.7 Taylor Series

Taylor Series Expansion

Symbols / Details:

- $f^{(n)}(a)$ – the n -th derivative of f evaluated at point a
- $n!$ – n factorial

$$f(x) = f(a) + f'(a)(x - a) + \frac{f''(a)}{2!}(x - a)^2 + \frac{f'''(a)}{3!}(x - a)^3 + \dots$$

In Words: Approximate any smooth function near a point a by adding up its value, slope, curvature, and higher-order terms at that point.

ML Use Case: A first-order Taylor expansion (just the linear term) is the mathematical justification for why a small gradient step reduces the loss; a second-order expansion is used to derive Newton's method.

Worked Example: **Step by step:** approximate $f(x) = e^x$ near $a = 0$ up to the quadratic term. Since $f(0) = 1$, $f'(0) = 1$, $f''(0) = 1$:

$$e^x \approx 1 + x + \frac{x^2}{2}$$

At $x = 0.2$: approximation = $1 + 0.2 + 0.02 = 1.22$, versus true $e^{0.2} \approx 1.2214$ – very close.

1.8 Integrals

Indefinite Integral (Antiderivative)

Symbols / Details:

- $f(x)$ – the function being integrated (the integrand)
- $F(x)$ – an antiderivative of f , i.e. $F'(x) = f(x)$
- C – constant of integration

$$\int f(x) dx = F(x) + C$$

In Words: Find a function whose derivative gives you back $f(x)$.

ML Use Case: Integration is how probability density functions are converted into actual probabilities, and how continuous expectation is computed.

Worked Example: $\int 2x dx = x^2 + C$ (check: $\frac{d}{dx}[x^2 + C] = 2x$ ✓).

Definite Integral

Symbols / Details:

- a, b – lower and upper limits of integration

$$\int_a^b f(x) dx = F(b) - F(a)$$

In Words: The exact area between the curve $f(x)$ and the x -axis, from $x = a$ to $x = b$.

ML Use Case: Used to compute the total probability under a probability density function (e.g. $\int_{-\infty}^{\infty} p(x) dx = 1$), and in computing continuous expectations.

Worked Example: $\int_0^2 x dx = \left[\frac{x^2}{2} \right]_0^2 = \frac{4}{2} - 0 = 2$ – the area of the triangle under the line $y = x$ from 0 to 2.

1.9 Optimization – Where Calculus Meets ML

Gradient Descent Update Rule

Symbols / Details:

- θ – model parameters (weights)
- η – learning rate
- $\nabla_{\theta} L(\theta)$ – gradient of the loss with respect to θ

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

In Words: Take the current parameters, move a small step (η) in the direction opposite to the gradient (because the gradient points uphill, and we want to go downhill toward lower loss).

ML Use Case: This single line of math is literally the update step executed every iteration of training in SGD, Adam, RMSProp, and virtually every neural network optimizer.

Worked Example: **Step by step:** let $L(\theta) = \theta^2$, start at $\theta_0 = 5$, learning rate $\eta = 0.1$.

$$\nabla_{\theta} L(\theta_0) = 2\theta_0 = 10$$

$$\theta_1 = 5 - 0.1(10) = 5 - 1 = 4$$

Repeat: $\nabla L(4) = 8 \Rightarrow \theta_2 = 4 - 0.1(8) = 3.2$. Each step moves θ closer to the minimum at $\theta = 0$.

Newton’s Method (root/optimization step)**Symbols / Details:**

- H^{-1} – inverse of the Hessian matrix

$$\theta_{t+1} = \theta_t - H^{-1} \nabla_{\theta} L(\theta_t)$$

In Words: Instead of a fixed step size, use the curvature (Hessian) to figure out exactly how far to step to reach the minimum of a locally quadratic approximation of the loss.

ML Use Case: Basis of second-order optimization methods (L-BFGS) which converge faster than plain gradient descent but are more expensive per step.

Worked Example: For $L(\theta) = \theta^2$: $\nabla L = 2\theta$, $H = 2$. One Newton step from $\theta_0 = 5$: $\theta_1 = 5 - \frac{1}{2}(10) = 0$ – it jumps straight to the minimum in a single step, because L is exactly quadratic.

Lagrange Multipliers (constrained optimization)**Symbols / Details:**

- $f(\mathbf{x})$ – objective function to optimize
- $g(\mathbf{x}) = 0$ – an equality constraint
- λ – the Lagrange multiplier

$$\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda g(\mathbf{x})$$

In Words: Combine the objective and constraint into one new function; at the optimum, the gradients of f and g must point in the same (or opposite) direction.

ML Use Case: Used to derive Support Vector Machines (maximizing margin subject to classification constraints) and constrained regularization problems.

Worked Example: Minimize $f(x, y) = x^2 + y^2$ subject to $x + y = 1$. Here $g(x, y) = x + y - 1$, so $\mathcal{L} = x^2 + y^2 + \lambda(x + y - 1)$. Setting $\partial \mathcal{L} / \partial x = 2x + \lambda = 0$ and $\partial \mathcal{L} / \partial y = 2y + \lambda = 0$ gives $x = y$, and with $x + y = 1$ we get $x = y = 0.5$ – the closest point to the origin on that line.

Convexity Condition**Symbols / Details:**

- f – a twice-differentiable function
- H – Hessian of f

$$f \text{ is convex} \iff H \succeq 0$$

In Words: A function is “bowl-shaped” (convex) if its curvature is never negative in any direction – meaning it has no false local minima to get stuck in.

ML Use Case: Guarantees that gradient descent on losses like linear/logistic regression (which are convex) will always find the global minimum.

Worked Example: For $f(x) = x^2$: $H = f''(x) = 2 \geq 0$ everywhere, so f is convex. For $f(x) = -x^2$: $H = -2 < 0$, so it is concave (not convex) – gradient descent could get stuck near a

maximum instead of finding a true minimum.

2 Part II — Linear Algebra

Linear algebra is the language data is written in. Every image, sentence, and dataset becomes a vector or matrix, and every neural network layer is fundamentally a matrix operation.

Symbol Legend – Linear Algebra

- $\mathbf{a}, \mathbf{b}, \mathbf{v}$ – lowercase bold letters = vectors
- A, B, M – uppercase letters = matrices
- a_i – the i -th component of vector $\mathbf{a}$
- A_{ij} – the entry in row i , column j of matrix A
- $A^\top$ – transpose of A (rows and columns swapped)
- A^{-1} – inverse of matrix A
- $\|\mathbf{v}\|$ – norm (length/magnitude) of vector $\mathbf{v}$
- $\cdot$ – dot product
- $\times$ – cross product (vectors) or ordinary multiplication (scalars)
- I – the identity matrix
- $\det(A)$ – determinant of A
- λ – an eigenvalue
- $\mathbf{x}$ – an eigenvector (when paired with λ)

2.1 Vectors – Basic Operations

Vector Addition

Symbols / Details:

- $\mathbf{a}, \mathbf{b}$ – two vectors of the same dimension

$$\mathbf{a} + \mathbf{b} = (a_1 + b_1, a_2 + b_2, \dots, a_n + b_n)$$

In Words: Add the vectors component by component – the first entries add together, the second entries add together, and so on.

ML Use Case: Used everywhere: combining feature vectors, adding bias vectors to layer outputs ($y = Wx + b$), residual/skip connections in ResNets and Transformers.

Worked Example: $(1, 2, 3) + (4, -1, 0) = (1+4, 2-1, 3+0) = (5, 1, 3)$.

Scalar Multiplication

Symbols / Details:

- c – a scalar (single number)
- $\mathbf{v}$ – a vector

$$c \mathbf{v} = (cv_1, cv_2, \dots, cv_n)$$

In Words: Multiply every component of the vector by the same number c , stretching or shrinking (and flipping, if $c < 0$) the vector.

ML Use Case: Learning rate scaling of gradients ($\eta \nabla L$) is exactly scalar multiplication.

Worked Example: $3 \cdot (1, -2, 4) = (3, -6, 12)$.

Dot Product (Inner Product)

Symbols / Details:

- $\mathbf{a} \cdot \mathbf{b}$ – the dot product, a single scalar number
- θ – the angle between the two vectors

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$$

In Words: Multiply corresponding components together and add up all the results. Equivalently, it equals the product of the two vectors' lengths times the cosine of the angle between them.

ML Use Case: The single most-used operation in ML: every neuron computes a dot product of its inputs with its weights ($\mathbf{w} \cdot \mathbf{x}$) before adding a bias.

Worked Example: Step by step: $\mathbf{a} = (1, 2, 3)$, $\mathbf{b} = (4, 0, -1)$.

$$\mathbf{a} \cdot \mathbf{b} = (1)(4) + (2)(0) + (3)(-1) = 4 + 0 - 3 = 1$$

Cross Product (3D only)

Symbols / Details:

- $\mathbf{a} \times \mathbf{b}$ – a new vector perpendicular to both $\mathbf{a}$ and $\mathbf{b}$

$$\mathbf{a} \times \mathbf{b} = (a_2 b_3 - a_3 b_2, a_3 b_1 - a_1 b_3, a_1 b_2 - a_2 b_1)$$

In Words: Produces a new 3D vector that is perpendicular to both original vectors, with a length equal to the area of the parallelogram they form.

ML Use Case: Rare in mainstream ML (more common in 3D graphics / robotics / physics-informed ML for surface normals and orientation).

Worked Example: $(1, 0, 0) \times (0, 1, 0) = (0 \cdot 0 - 0 \cdot 1, 0 \cdot 0 - 1 \cdot 0, 1 \cdot 1 - 0 \cdot 0) = (0, 0, 1)$.

Vector Norm – General L_p Norm

Symbols / Details:

- p – the order of the norm
- $\|\mathbf{v}\|_p$ – the p -norm of $\mathbf{v}$

$$\|\mathbf{v}\|_p = \left(\sum_{i=1}^n |v_i|^p \right)^{1/p}$$

In Words: Raise every component to the power p , add them up, then take the p -th root – a general way of measuring vector “length.”

ML Use Case: Generalizes the specific norms below, which appear throughout regularization and distance metrics.

Worked Example: For $\mathbf{v} = (3, -4)$ and $p = 3$: $\|\mathbf{v}\|_3 = (27 + 64)^{1/3} = 91^{1/3} \approx 4.50$.

L1 Norm (Manhattan Norm)

Symbols / Details:

- $\|\mathbf{v}\|_1$ – sum of absolute values

$$\|\mathbf{v}\|_1 = \sum_{i=1}^n |v_i|$$

In Words: Add up the absolute values of every component – like walking city blocks instead of cutting diagonally.

ML Use Case: Used in Lasso (L1) regularization, which pushes many weights exactly to zero, producing sparse models.

Worked Example: $\mathbf{v} = (3, -4)$: $\|\mathbf{v}\|_1 = |3| + |-4| = 7$.

L2 Norm (Euclidean Norm)

Symbols / Details:

- $\|\mathbf{v}\|_2$ – ordinary straight-line length

$$\|\mathbf{v}\|_2 = \sqrt{\sum_{i=1}^n v_i^2} = \sqrt{\mathbf{v} \cdot \mathbf{v}}$$

In Words: Square every component, sum them, then take the square root – the ordinary straight-line (“as the crow flies”) length of the vector.

ML Use Case: Used in Ridge (L2) regularization, weight decay, computing Euclidean distance in k-NN and k-means, and gradient-norm clipping.

Worked Example: $\mathbf{v} = (3, -4)$: $\|\mathbf{v}\|_2 = \sqrt{9 + 16} = \sqrt{25} = 5$.

Unit Vector (Normalization)

Symbols / Details:

- $\hat{\mathbf{v}}$ – the unit vector in the direction of $\mathbf{v}$

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}$$

In Words: Divide the vector by its own length so the result always has length exactly 1, keeping only its direction.

ML Use Case: Feature/embedding normalization before computing cosine similarity, and normalizing gradients in optimizers.

Worked Example: $\mathbf{v} = (3, -4)$, $\|\mathbf{v}\|_2 = 5 \Rightarrow \hat{\mathbf{v}} = (3/5, -4/5) = (0.6, -0.8)$, which indeed has length $\sqrt{0.36 + 0.64} = 1$.

Cosine Similarity

Symbols / Details:

- $\mathbf{a}, \mathbf{b}$ – two vectors being compared
- $\cos \theta$ – cosine of the angle between them, range $[-1, 1]$

$$\text{cosine similarity}(\mathbf{a}, \mathbf{b}) = \cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

In Words: Take the dot product of the two vectors and divide by the magnitude of $\mathbf{a}$ times the magnitude of $\mathbf{b}$. The result measures how similarly directed two vectors are, ignoring their length.

ML Use Case: The standard way to compare word/sentence embeddings, recommendation-system item vectors, and document similarity in NLP and search.

Worked Example: **Step by step:** $\mathbf{a} = (1, 2, 3)$, $\mathbf{b} = (4, 0, -1)$. From before, $\mathbf{a} \cdot \mathbf{b} = 1$.

$$\|\mathbf{a}\| = \sqrt{1 + 4 + 9} = \sqrt{14} \approx 3.742, \quad \|\mathbf{b}\| = \sqrt{16 + 0 + 1} = \sqrt{17} \approx 4.123$$

$$\cos \theta = \frac{1}{3.742 \times 4.123} \approx \frac{1}{15.43} \approx 0.065$$

Nearly 0 means the vectors point in almost perpendicular (unrelated) directions.

Euclidean Distance

Symbols / Details:

- $d(\mathbf{a}, \mathbf{b})$ – straight-line distance between two points/vectors

$$d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_2 = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

In Words: Subtract the vectors component-wise, then take the L2 norm of the result – the straight-line distance between the two points they represent.

ML Use Case: Core distance metric in k-Nearest Neighbors, k-Means clustering, and anomaly detection.

Worked Example: $\mathbf{a} = (1, 2)$, $\mathbf{b} = (4, 6)$: $d = \sqrt{(1-4)^2 + (2-6)^2} = \sqrt{9 + 16} = \sqrt{25} = 5$.

2.2 Matrices – Basic Operations

Matrix Addition

Symbols / Details:

- A, B – matrices of identical shape ($m \times n$)

$$(A + B)_{ij} = A_{ij} + B_{ij}$$

In Words: Add matrices entry-by-entry, position matching position.

ML Use Case: Combining gradients accumulated from different mini-batches, or summing weight updates.

Worked Example: $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 6 & 2 \\ 4 & 5 \end{bmatrix}$.

Matrix Multiplication

Symbols / Details:

- A – an ($m \times n$) matrix
- B – an ($n \times p$) matrix
- $(AB)_{ij}$ – entry at row i , column j of the product

$$(AB)_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$$

In Words: To get entry (i, j) of the product, take the dot product of row i of A with column j of B . The inner dimensions (n) must match.

ML Use Case: This is the single most executed operation in deep learning: every dense/linear layer computes $y = Wx$, and GPUs are essentially matrix-multiplication machines.

Worked Example: Step by step: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}$.

$$(AB)_{11} = (1)(2) + (2)(1) = 4, \quad (AB)_{12} = (1)(0) + (2)(2) = 4$$

$$(AB)_{21} = (3)(2) + (4)(1) = 10, \quad (AB)_{22} = (3)(0) + (4)(2) = 8$$

$$AB = \begin{bmatrix} 4 & 4 \\ 10 & 8 \end{bmatrix}$$

Matrix Transpose

Symbols / Details:

- $A^\top$ – transpose of A

$$(A^\top)_{ij} = A_{ji}$$

In Words: Flip the matrix over its diagonal – rows become columns and columns become rows.

ML Use Case: Used in computing $X^\top X$ for the Normal Equation in linear regression, and to align dimensions for matrix multiplication in backprop.

Worked Example: $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \Rightarrow A^\top = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$

Identity Matrix

Symbols / Details:

- I – identity matrix, 1's on the diagonal, 0's elsewhere

$$AI = IA = A$$

In Words: Multiplying any matrix by the identity matrix leaves it completely unchanged – it is the matrix equivalent of multiplying a number by 1.

ML Use Case: Appears in regularized regression ($X^\top X + \lambda I$) and as the starting point for weight initialization schemes.

Worked Example: $\begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix}$ – unchanged.

Matrix Inverse

Symbols / Details:

- A^{-1} – the inverse of a square matrix A

$$AA^{-1} = A^{-1}A = I$$

In Words: The inverse is the matrix that “undoes” A : multiplying them together (in either order) gives back the identity matrix, just like $x \cdot x^{-1} = 1$ for numbers.

ML Use Case: Used in the closed-form solution of linear regression (the Normal Equation) and in whitening/decorrelating data.

Worked Example: **Step by step:** for $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$, $A^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$. For $A = \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}$:
 $\det = 4 - 0 = 4$, so

$$A^{-1} = \frac{1}{4} \begin{bmatrix} 2 & 0 \\ -1 & 2 \end{bmatrix} = \begin{bmatrix} 0.5 & 0 \\ -0.25 & 0.5 \end{bmatrix}$$

Check: $AA^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \checkmark$

Determinant (2×2 case)

Symbols / Details:

- $\det(A)$ – a single scalar describing how A scales area/volume

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$$

In Words: A number that tells you how much a matrix stretches or shrinks space; if it is zero, the matrix squashes space into a lower dimension (it is not invertible).

ML Use Case: Used to check whether a covariance matrix is invertible (needed for the multivariate Gaussian density), and appears in the normalizing constant of that distribution.

Worked Example: $\det \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix} = (2)(2) - (0)(1) = 4$. Nonzero, so this matrix is invertible.

Trace

Symbols / Details:

- $\text{tr}(A)$ – sum of diagonal entries

$$\text{tr}(A) = \sum_{i=1}^n A_{ii}$$

In Words: Add up only the diagonal entries of a square matrix.

ML Use Case: Used in computing the Frobenius norm, in some regularization terms, and in the formula for variance of a linear combination via covariance matrices.

Worked Example: $\text{tr} \begin{bmatrix} 2 & 5 \\ 1 & 6 \end{bmatrix} = 2 + 6 = 8$.

Rank

Symbols / Details:

- $\text{rank}(A)$ – number of linearly independent rows/columns

$$\text{rank}(A) = \dim(\text{column space of } A)$$

In Words: Counts how many “truly independent directions” of information the matrix actually contains – redundant rows/columns don’t add to the rank.

ML Use Case: Low-rank approximation (via SVD) is used to compress neural-network weight matrices and in recommender systems (matrix factorization).

Worked Example: $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$: row 2 is exactly $2 \times$ row 1, so the rows are not independent $\Rightarrow \text{rank}(A) = 1$, even though A is 2×2 .

2.3 Eigenvalues, Eigenvectors, and Decompositions

Eigenvalue Equation

Symbols / Details:

- A – a square matrix
- $\mathbf{x}$ – an eigenvector of A (nonzero)
- λ – the corresponding eigenvalue (a scalar)

$$A\mathbf{x} = \lambda\mathbf{x}$$

In Words: When matrix A acts on its eigenvector $\mathbf{x}$, the direction of $\mathbf{x}$ does not change – it is only stretched or shrunk by the factor λ .

ML Use Case: Principal Component Analysis (PCA) finds the eigenvectors of the covariance matrix; the eigenvalues tell you how much variance each principal component explains.

Worked Example: Step by step: $A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$. Solve $\det(A - \lambda I) = 0$:

$$(2 - \lambda)(3 - \lambda) = 0 \Rightarrow \lambda_1 = 2, \lambda_2 = 3$$

For $\lambda_1 = 2$: solve $(A - 2I)\mathbf{x} = 0 \Rightarrow \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \mathbf{x} = 0$, giving eigenvector $\mathbf{x}_1 = (1, 0)^\top$. For $\lambda_2 = 3$: eigenvector $\mathbf{x}_2 = (0, 1)^\top$. Check: $A\mathbf{x}_1 = (2, 0)^\top = 2\mathbf{x}_1 \checkmark$

Eigendecomposition

Symbols / Details:

- V – matrix whose columns are the eigenvectors of A
- Λ – diagonal matrix of eigenvalues

$$A = V\Lambda V^{-1}$$

In Words: Rewrite A as: rotate into the eigenvector basis (V^{-1}), simply scale along each axis (Λ), then rotate back (V).

ML Use Case: Diagonalizing a covariance matrix this way is exactly how PCA computes its principal axes.

Worked Example: Continuing the example above: $V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, $\Lambda = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$. Since A was already diagonal, $V = I$ and $A = V\Lambda V^{-1} = \Lambda$ directly – the eigenvectors already line up with the coordinate axes.

Singular Value Decomposition (SVD)

Symbols / Details:

- A – any ($m \times n$) matrix (need not be square)
- U, V – orthogonal matrices (left/right singular vectors)
- Σ – diagonal matrix of singular values (≥ 0)

$$A = U\Sigma V^\top$$

In Words: Any matrix, however irregular, can be broken into a rotation ($V^\top$), a pure axis-aligned scaling (Σ), and another rotation (U).

ML Use Case: Backbone of dimensionality reduction (truncated SVD), recommender systems (matrix factorization of the user-item matrix), and computing PCA in practice.

Worked Example: For a diagonal matrix like $A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$, the SVD is trivial: $U = I$, $\Sigma = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}$ (singular values sorted largest first), $V = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$. For a general (non-diagonal) matrix, Σ 's entries are the square roots of the eigenvalues of $A^\top A$.

Positive (Semi-)Definite Matrix

Symbols / Details:

- M – a symmetric matrix
- $\mathbf{x}^\top M \mathbf{x}$ – a scalar, the “quadratic form” of M

$$M \succeq 0 \iff \mathbf{x}^\top M \mathbf{x} \geq 0 \text{ for all } \mathbf{x}$$

In Words: A matrix is positive semi-definite if it never produces a negative number when sandwiched between any vector and its transpose – geometrically, it never “flips” any direction.

ML Use Case: Every valid covariance matrix must be positive semi-definite; the Hessian being positive semi-definite is exactly the condition for a convex loss function.

Worked Example: Test $M = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$ with $\mathbf{x} = (x_1, x_2)$: $\mathbf{x}^\top M \mathbf{x} = 2x_1^2 + 3x_2^2 \geq 0$ always (sum of squares with positive coefficients) $\Rightarrow M \succeq 0$.

Orthogonal Projection onto a Vector

Symbols / Details:

- $\text{proj}_{\mathbf{b}} \mathbf{a}$ – the projection of $\mathbf{a}$ onto $\mathbf{b}$

$$\text{proj}_{\mathbf{b}} \mathbf{a} = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{b}\|^2} \mathbf{b}$$

In Words: Find the “shadow” that vector $\mathbf{a}$ casts onto the line defined by vector $\mathbf{b}$.

ML Use Case: Used in Gram–Schmidt orthogonalization and in deriving least-squares regression geometrically (projecting the target vector onto the column space of the feature matrix).

Worked Example: **Step by step:** $\mathbf{a} = (3, 4)$, $\mathbf{b} = (1, 0)$.

$$\mathbf{a} \cdot \mathbf{b} = 3, \quad \|\mathbf{b}\|^2 = 1 \Rightarrow \text{proj}_{\mathbf{b}} \mathbf{a} = \frac{3}{1}(1, 0) = (3, 0)$$

The shadow of $(3, 4)$ onto the x -axis is $(3, 0)$, as expected.

3 Part III — Probability

Probability is the mathematics of uncertainty. Every model prediction that comes with a confidence score, every generative model, and every loss function derived from “likelihood” is built on probability theory.

Symbol Legend – Probability

- $P(A)$ – probability of event A , a number between 0 and 1
- $P(A \cap B)$ – probability that both A and B happen (joint probability)
- $P(A \cup B)$ – probability that A or B (or both) happens
- $P(A | B)$ – probability of A given that B has happened (conditional)
- X – a random variable
- $p(x)$ – probability mass/density function of X , evaluated at value x
- $F(x)$ – cumulative distribution function
- $E[X], \mu$ – expectation (mean) of X
- $\text{Var}(X), \sigma^2$ – variance of X
- $\sim$ – “is distributed as”, e.g. $X \sim \mathcal{N}(\mu, \sigma^2)$
- θ – a parameter of a distribution/model

3.1 Foundations

Axioms of Probability

Symbols / Details:

- Ω – the sample space (set of all possible outcomes)

$$0 \leq P(A) \leq 1, \quad P(\Omega) = 1, \quad P(A \cup B) = P(A) + P(B) \text{ if } A, B \text{ disjoint}$$

In Words: Every probability is between 0 and 1, something in the sample space is guaranteed to happen, and probabilities of non-overlapping events simply add together.

ML Use Case: These three rules are the bedrock every ML probabilistic model (Naive Bayes, HMMs, Bayesian Networks, diffusion models) must respect.

Worked Example: Rolling a fair die: $\Omega = \{1, 2, 3, 4, 5, 6\}$. $P(\Omega) = 6 \times \frac{1}{6} = 1$. Events “roll = 2” and “roll = 5” are disjoint, so $P(2 \text{ or } 5) = \frac{1}{6} + \frac{1}{6} = \frac{1}{3}$.

Complement Rule

Symbols / Details:

- A^c – “not A ” – the complement of event A

$$P(A^c) = 1 - P(A)$$

In Words: The probability that an event does NOT happen is one minus the probability that it

does.

ML Use Case: Used to compute false-negative / false-positive style probabilities and in evaluating classifier error rates.

Worked Example: If $P(\text{spam}) = 0.3$, then $P(\text{not spam}) = 1 - 0.3 = 0.7$.

Addition Rule (Union)

Symbols / Details:

- A, B – two (possibly overlapping) events

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

In Words: Add the individual probabilities, then subtract the overlap once so it isn't double-counted.

ML Use Case: Used in computing probability of “at least one” of several model errors or events occurring.

Worked Example: $P(\text{rains}) = 0.4$, $P(\text{windy}) = 0.3$, $P(\text{both}) = 0.1$. $P(\text{rains or windy}) = 0.4 + 0.3 - 0.1 = 0.6$.

Joint Probability

Symbols / Details:

- $P(A, B)$ – same as $P(A \cap B)$ – probability both happen

$$P(A, B) = P(A | B) P(B) = P(B | A) P(A)$$

In Words: The chance of two things happening together equals the chance of one happening, times the chance of the other given the first already happened.

ML Use Case: The foundation of the “chain rule of probability” used to factorize joint distributions in language models: $P(w_1, \dots, w_n) = \prod_t P(w_t | w_{<t})$.

Worked Example: $P(\text{rain}) = 0.3$, $P(\text{traffic} | \text{rain}) = 0.8$. $P(\text{rain and traffic}) = 0.3 \times 0.8 = 0.24$.

Marginal Probability

Symbols / Details:

- $P(A)$ – probability of A regardless of B

$$P(A) = \sum_b P(A, B=b)$$

In Words: “Sum out” (or integrate out, for continuous variables) the other variable to get the probability of just the one you care about.

ML Use Case: Used to compute marginal likelihoods in Bayesian models and to obtain a single feature's distribution from a joint dataset distribution.

Worked Example: If $P(\text{rain, weekday}) = 0.2$ and $P(\text{rain, weekend}) = 0.1$, then $P(\text{rain}) = 0.2 + 0.1 = 0.3$.

Conditional Probability

Symbols / Details:

- $P(A | B)$ – probability of A given that B is known to have occurred

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0$$

In Words: Restrict attention to only the cases where B happened, then ask what fraction of those cases also have A .

ML Use Case: This exact quantity is what a classifier outputs: $P(\text{class} | \text{features})$.

Worked Example: Step by step: in a deck of 52 cards, let A = “card is a King”, B = “card is a face card” (12 face cards: J, Q, K $\times$ 4 suits).

$$P(A \cap B) = P(\text{King}) = \frac{4}{52}, \quad P(B) = \frac{12}{52}$$

$$P(A | B) = \frac{4/52}{12/52} = \frac{4}{12} = \frac{1}{3}$$

Knowing it’s a face card, there’s a $1/3$ chance it’s specifically a King.

Bayes’ Theorem

Symbols / Details:

- $P(A | B)$ – posterior – updated belief about A after seeing B
- $P(B | A)$ – likelihood – how probable the evidence is if A is true
- $P(A)$ – prior – belief about A before seeing evidence
- $P(B)$ – evidence – total probability of observing B

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}$$

In Words: Update your belief in A by combining what you believed before (the prior) with how well the new evidence fits that belief (the likelihood), then rescale by the overall chance of seeing that evidence.

ML Use Case: Directly powers Naive Bayes classifiers, Bayesian inference, spam filters, and the theoretical foundation of Bayesian deep learning.

Worked Example: Step by step (classic medical test example): a disease affects 1% of people ($P(D) = 0.01$). A test is 90% sensitive ($P(+ | D) = 0.9$) and has a 5% false-positive rate ($P(+ | D^c) = 0.05$). Given a positive test, what’s $P(D | +)$?

$$P(+) = P(+ | D)P(D) + P(+ | D^c)P(D^c) = (0.9)(0.01) + (0.05)(0.99) = 0.009 + 0.0495 = 0.0585$$

$$P(D | +) = \frac{(0.9)(0.01)}{0.0585} = \frac{0.009}{0.0585} \approx 0.154$$

Even with a positive test, there's only about a 15.4% chance of actually having the disease – because the disease is rare, false positives dominate.

Independence

Symbols / Details:

- $A \perp B$ – “ A is independent of B ”

$$P(A \cap B) = P(A)P(B) \iff A \perp B$$

In Words: Two events are independent exactly when knowing one happened tells you nothing about the other – their joint probability is just the product of the individual ones.

ML Use Case: The “i.i.d.” (independent and identically distributed) assumption behind almost every ML training-data assumption relies on this definition.

Worked Example: Two fair coin flips: $P(1st = H) = 0.5$, $P(2nd = H) = 0.5$. $P(\text{both } H) = 0.5 \times 0.5 = 0.25$, matching the independence formula – confirming the flips don't affect each other.

3.2 Random Variables and Distributions

Probability Mass Function (PMF) – discrete

Symbols / Details:

- $p(x)$ – probability that discrete X takes exact value x

$$p(x) = P(X = x), \quad \sum_x p(x) = 1$$

In Words: For a discrete/countable random variable, the PMF gives the exact probability of each possible outcome, and all outcomes' probabilities must add to 1.

ML Use Case: Used for classification outputs, where the softmax layer produces a PMF over class labels.

Worked Example: Fair die: $p(x) = \frac{1}{6}$ for $x \in \{1, \dots, 6\}$. Check total: $6 \times \frac{1}{6} = 1$ ✓

Probability Density Function (PDF) – continuous

Symbols / Details:

- $p(x)$ – density at x (NOT a probability itself)

$$P(a \leq X \leq b) = \int_a^b p(x) dx, \quad \int_{-\infty}^{\infty} p(x) dx = 1$$

In Words: For continuous variables, probability only makes sense over a range; the density function's area under a stretch of the curve gives that range's probability, and the total area under

the whole curve is 1.

ML Use Case: Underlies every continuous generative model (VAEs, diffusion models, normalizing flows) and Gaussian noise models.

Worked Example: For a Uniform(0, 10) variable, $p(x) = 0.1$ everywhere on $[0, 10]$. $P(2 \leq X \leq 5) = \int_2^5 0.1 dx = 0.1 \times 3 = 0.3$.

Cumulative Distribution Function (CDF)

Symbols / Details:

- $F(x)$ – probability that X is less than or equal to x

$$F(x) = P(X \leq x) = \int_{-\infty}^x p(t) dt$$

In Words: Accumulate probability from the far left up to point x – it always increases from 0 to 1.

ML Use Case: Used for quantile computation, percentile-based evaluation metrics, and sampling via inverse-CDF methods.

Worked Example: For Uniform(0, 10): $F(x) = x/10$. $F(4) = 0.4$, meaning there's a 40% chance $X \leq 4$.

Expectation (Discrete)

Symbols / Details:

- $E[X], \mu$ – expected value / mean of X

$$E[X] = \sum_x x p(x)$$

In Words: A probability-weighted average: multiply each possible value by how likely it is, then add them all up.

ML Use Case: The expected loss over the training distribution is precisely what every ML loss function tries to approximate and minimize.

Worked Example: **Step by step:** fair die, $p(x) = 1/6$ for each face.

$$E[X] = \sum_{x=1}^6 x \cdot \frac{1}{6} = \frac{1 + 2 + 3 + 4 + 5 + 6}{6} = \frac{21}{6} = 3.5$$

Expectation (Continuous)

Symbols / Details:

- $E[X]$ – expected value of a continuous random variable

$$E[X] = \int_{-\infty}^{\infty} x p(x) dx$$

In Words: Same idea as the discrete case, but summing becomes integrating over a continuum of possible values.

ML Use Case: Used to define the population mean that a model's outputs try to approximate, and in policy-gradient reinforcement learning (expected reward).

Worked Example: For $X \sim \text{Uniform}(0, 10)$: by symmetry, $E[X] = \frac{0+10}{2} = 5$ (the midpoint).

Variance (via expectation)

Symbols / Details:

- $\text{Var}(X), \sigma^2$ – variance – spread of X around its mean

$$\text{Var}(X) = E[(X - \mu)^2] = E[X^2] - (E[X])^2$$

In Words: Average the squared distance of outcomes from the mean – how “spread out” the random variable's values tend to be.

ML Use Case: Used in uncertainty estimation, the variance term of the bias-variance decomposition, and batch normalization statistics.

Worked Example: **Step by step:** fair die, $\mu = 3.5$ (from above). $E[X^2] = \sum x^2 \cdot \frac{1}{6} = \frac{1+4+9+16+25+36}{6} = \frac{91}{6} \approx 15.17$.

$$\text{Var}(X) = 15.17 - (3.5)^2 = 15.17 - 12.25 \approx 2.92$$

Covariance (probabilistic)

Symbols / Details:

- $\text{Cov}(X, Y)$ – how two random variables vary together

$$\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$$

In Words: Measures whether X and Y tend to be above/below their means at the same time (positive), opposite times (negative), or unrelated (near zero).

ML Use Case: Builds the covariance matrix that PCA decomposes to find directions of maximum variance in the data.

Worked Example: If taller people tend to weigh more, $\text{Cov}(\text{height}, \text{weight}) > 0$ – when height is above its mean, weight tends to be above its mean too.

3.3 Common Probability Distributions

Bernoulli Distribution

Symbols / Details:

- p – probability of “success” (outcome = 1)

$$P(X = x) = p^x(1 - p)^{1-x}, \quad x \in \{0, 1\}$$

In Words: Models a single coin-flip-like event: success with probability p , failure with probability $1 - p$.

ML Use Case: The distribution behind binary classification – the sigmoid output of a logistic regression / neural-network binary classifier is the Bernoulli parameter p .

Worked Example: A biased coin with $p = 0.7$ for heads: $P(X=1) = 0.7$, $P(X=0) = 0.3^1 \cdot 0.7^0 = 0.3$.

Binomial Distribution

Symbols / Details:

- n – number of independent trials
- k – number of successes
- $\binom{n}{k}$ – “ n choose k ,” number of ways to pick k successes from n trials

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

In Words: The probability of getting exactly k successes out of n independent Bernoulli trials, each with success probability p .

ML Use Case: Used to model counts of successes (e.g. correct predictions out of n samples) and in some A/B testing statistical tests.

Worked Example: **Step by step:** flip a fair coin ($p = 0.5$) $n = 4$ times. Probability of exactly $k = 2$ heads:

$$\binom{4}{2} = 6, \quad P(X=2) = 6 \times (0.5)^2 (0.5)^2 = 6 \times 0.0625 = 0.375$$

Poisson Distribution

Symbols / Details:

- λ – average rate of events in a fixed interval
- k – observed number of events

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

In Words: Models the probability of observing exactly k events in a fixed window, given they happen at a known average rate λ and independently of each other.

ML Use Case: Used to model count data such as word occurrence counts or click/event counts in some NLP and recommendation models.

Worked Example: If a website gets $\lambda = 3$ signups per hour on average, probability of exactly

$k = 5$ signups in an hour:

$$P(X=5) = \frac{3^5 e^{-3}}{5!} = \frac{243 \times 0.0498}{120} \approx 0.101$$

Uniform Distribution (continuous)

Symbols / Details:

- a, b – lower and upper bounds of the interval

$$p(x) = \frac{1}{b-a}, \quad a \leq x \leq b$$

In Words: Every value in the interval $[a, b]$ is equally likely – a flat, constant density.

ML Use Case: Used for random weight initialization schemes and for uniform sampling in Monte Carlo methods.

Worked Example: Uniform(0, 4): $p(x) = \frac{1}{4-0} = 0.25$ for any $x \in [0, 4]$.

Gaussian / Normal Distribution

Symbols / Details:

- μ – mean
- σ^2 – variance
- $\mathcal{N}(\mu, \sigma^2)$ – shorthand for “Normal with mean μ , variance σ^2 ”

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

In Words: The classic “bell curve”: most probability mass sits near the mean μ , and it falls off symmetrically the farther you move away, at a rate controlled by σ .

ML Use Case: Assumed noise model behind Mean Squared Error loss (MSE is the negative log-likelihood under a Gaussian), weight initialization, and the reparameterization trick in VAEs.

Worked Example: For $\mathcal{N}(0, 1)$ (the standard normal) at $x = 0$: $p(0) = \frac{1}{\sqrt{2\pi}} e^0 = \frac{1}{2.5066} \approx 0.399$, the peak height of the bell curve.

Multivariate Gaussian Distribution

Symbols / Details:

- $\boldsymbol{\mu}$ – mean vector
- Σ – covariance matrix
- $|\Sigma|$ – determinant of the covariance matrix

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

In Words: The multi-dimensional bell curve: a point's density depends on its Mahalanobis distance from the mean vector, reshaped by the covariance matrix.

ML Use Case: Used in Gaussian Mixture Models, the latent-space prior of VAEs, and Gaussian Processes.

Worked Example: With $d = 2$, $\mu = (0, 0)$ and $\Sigma = I$ (independent unit-variance dimensions), the formula collapses to the product of two independent standard normal densities – an uncorrelated 2D bell surface centered at the origin.

Exponential Distribution

Symbols / Details:

- λ – rate parameter

$$p(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

In Words: Models the waiting time until the next event in a process where events occur continuously and independently at a constant rate.

ML Use Case: Used to model time-to-failure/time-to-event data in survival-analysis style ML applications.

Worked Example: If a server fails on average every $\lambda = 0.1$ per hour, probability density of failing exactly at $x = 5$ hours: $p(5) = 0.1 e^{-0.5} \approx 0.0607$.

3.4 Limit Theorems and Information Theory

Law of Large Numbers

Symbols / Details:

- $\bar{X}_n$ – sample average of n i.i.d. draws

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{n \rightarrow \infty} \mu$$

In Words: As you collect more and more samples, their average gets closer and closer to the true population mean.

ML Use Case: Justifies why averaging loss over a large mini-batch approximates the true expected loss over the whole data distribution.

Worked Example: Flip a fair coin ($\mu = 0.5$) many times: after 10 flips the average might be 0.6; after 10,000 flips it will almost certainly be very close to 0.5.

Central Limit Theorem

Symbols / Details:

- $\bar{X}_n$ – sample mean of n i.i.d. samples

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow[n \rightarrow \infty]{d} \mathcal{N}(0, 1)$$

In Words: No matter what distribution the original data comes from, the average of many independent samples becomes approximately Gaussian as the sample size grows.

ML Use Case: Justifies using normal-distribution-based confidence intervals and significance tests on model evaluation metrics.

Worked Example: Even if individual die rolls are uniform (not bell-shaped), the average of, say, 30 rolls will look approximately normal, centered at 3.5 with a narrower spread than a single roll.

Entropy

Symbols / Details:

- $H(X)$ – entropy – average “surprise” / uncertainty of X , in bits/nats

$$H(X) = - \sum_x p(x) \log p(x)$$

In Words: Weight each outcome’s “surprise” ($-\log p(x)$, higher when the outcome is rarer) by its probability, and average over all outcomes.

ML Use Case: Used to build decision-tree splitting criteria (information gain) and as the basis for cross-entropy loss.

Worked Example: **Step by step:** a fair coin, $p(H) = p(T) = 0.5$ (using $\log_2$):

$$H(X) = -[0.5 \log_2 0.5 + 0.5 \log_2 0.5] = -[0.5(-1) + 0.5(-1)] = 1 \text{ bit}$$

Maximum uncertainty for a coin – you need exactly 1 bit to describe the outcome.

Cross-Entropy

Symbols / Details:

- p – true distribution (ground-truth labels)
- q – predicted distribution (model output)

$$H(p, q) = - \sum_x p(x) \log q(x)$$

In Words: Measures how many extra “bits of surprise” you incur, on average, if you believe the model’s predicted distribution q when the true distribution is actually p .

ML Use Case: THE most common classification loss function in deep learning – `CrossEntropyLoss` in PyTorch is exactly this formula.

Worked Example: **Step by step:** true label is class A, so $p = (1, 0)$ (one-hot). Model predicts $q = (0.8, 0.2)$.

$$H(p, q) = -[1 \cdot \log(0.8) + 0 \cdot \log(0.2)] = -\log(0.8) \approx 0.223 \text{ nats}$$

If the model had instead predicted $q = (0.99, 0.01)$ (more confident and correct), the loss would drop to $-\log(0.99) \approx 0.01$ – much lower.

Kullback–Leibler (KL) Divergence

Symbols / Details:

- $D_{KL}(p||q)$ – “distance” from distribution q to distribution p (not symmetric)

$$D_{KL}(p||q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = H(p, q) - H(p)$$

In Words: Measures how different one probability distribution is from another, in units of extra “surprise” incurred by using the wrong distribution q instead of the true one p .

ML Use Case: The regularization term in VAEs (keeping the learned latent distribution close to a prior), and used in knowledge distillation and reinforcement learning (policy regularization).

Worked Example: Continuing the cross-entropy example: $p = (1, 0)$ is a one-hot label, so $H(p) = -1 \log 1 - 0 = 0$. From above, $H(p, q) \approx 0.223$.

$$D_{KL}(p||q) = H(p, q) - H(p) = 0.223 - 0 = 0.223 \text{ nats}$$

Here KL divergence equals cross-entropy exactly, because a one-hot true distribution has zero entropy of its own.

3.5 Estimation

Maximum Likelihood Estimation (MLE)

Symbols / Details:

- θ – model parameters to estimate
- $L(\theta)$ – likelihood of the parameters given observed data

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \prod_{i=1}^n p(x_i | \theta) = \arg \max_{\theta} \sum_{i=1}^n \log p(x_i | \theta)$$

In Words: Find the parameter values that make the observed data as probable as possible – in practice, maximize the sum of log-probabilities (log-likelihood) instead of the product, for numerical stability.

ML Use Case: Nearly every standard ML loss function (MSE, cross-entropy) is derived by taking the negative log-likelihood under some assumed distribution and minimizing it – this is MLE in disguise.

Worked Example: **Step by step:** flip a coin 10 times, observe 7 heads. For Bernoulli(p), the log-likelihood is $\ell(p) = 7 \log p + 3 \log(1 - p)$. Setting $\frac{d\ell}{dp} = \frac{7}{p} - \frac{3}{1-p} = 0$ gives $7(1 - p) = 3p \Rightarrow p = 0.7$. The MLE estimate is simply the observed proportion of heads, $\hat{p} = 7/10 = 0.7$.

Maximum A Posteriori (MAP) Estimation

Symbols / Details:

- $p(\theta)$ – prior distribution over parameters

$$\hat{\theta}_{MAP} = \arg \max_{\theta} p(\theta \mid \text{data}) = \arg \max_{\theta} \left[\log p(\text{data} \mid \theta) + \log p(\theta) \right]$$

In Words: Like MLE, but also folds in a prior belief about which parameter values are plausible before seeing the data.

ML Use Case: Adding a Gaussian prior on the weights and doing MAP estimation is mathematically identical to L2/Ridge regularization – this is where “weight decay” comes from.

Worked Example: Same coin example, but now with a prior belief that the coin is probably close to fair (a prior peaked at $p = 0.5$). MAP will pull the estimate from the MLE value of 0.7 slightly back toward 0.5 – e.g. landing around $\hat{p} \approx 0.65$, depending on how strong the prior is.

4 Part IV — Statistics

Statistics turns raw numbers into insight: summarizing data (descriptive statistics), quantifying its shape, and deciding whether a pattern is real or just noise (inferential statistics). This is also where basic summary measures like mean, median, mode, skewness, and kurtosis live – they are statistics tools, not calculus/probability tools, even though they lean on both.

Symbol Legend – Statistics

- x_i – the i -th data point in a sample
- n – sample size (number of data points)
- $\bar{x}$ – sample mean
- μ – population mean
- s, s^2 – sample standard deviation, sample variance
- σ, σ^2 – population standard deviation, population variance
- r – sample (Pearson) correlation coefficient
- H_0 – null hypothesis
- α – significance level (e.g. 0.05)
- $\hat{y}$ – model’s predicted value
- y – true/observed value

4.1 Descriptive Statistics – Measures of Central Tendency

Mean (Average)

Symbols / Details:

- $\bar{x}$ – sample mean
- x_i – each individual data point
- n – number of data points

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

In Words: Add up every value in the dataset, then divide by how many values there are.

ML Use Case: The baseline “dumbest possible prediction” a model is compared against; also what Batch Normalization centers activations around.

Worked Example: Step by step – data: 4, 8, 6 (so $n = 3$):

$$\bar{x} = \frac{1}{3} \sum_{i=1}^3 x_i = \frac{x_1 + x_2 + x_3}{3} = \frac{4 + 8 + 6}{3} = \frac{18}{3} = 6$$

So the mean of the dataset $\{4, 8, 6\}$ is **6**.

Median**Symbols / Details:**

- $\tilde{x}$ – the median – the middle value

$$\tilde{x} = \begin{cases} x_{(\frac{n+1}{2})} & n \text{ odd} \\ \frac{1}{2}(x_{(n/2)} + x_{(n/2+1)}) & n \text{ even} \end{cases}$$

In Words: Sort all the data from smallest to largest, and pick the value that sits exactly in the middle (or average the two middle values if there's an even number of points).

ML Use Case: More robust to outliers than the mean; used in robust-statistics-based preprocessing and in evaluation metrics like Median Absolute Error.

Worked Example: **Step by step – data:** 4, 8, 6: sort to get 4, 6, 8 ($n = 3$, odd). Middle position = $\frac{3+1}{2} = 2$, so $\tilde{x} = 6$ (the 2nd sorted value).

If we add a point, data 4, 6, 8, 20 ($n = 4$, even): sorted is 4, 6, 8, 20, median = $\frac{6+8}{2} = 7$.

Mode**Symbols / Details:**

- Mode – the most frequently occurring value(s)

$$\text{Mode} = \arg \max_x (\text{count of } x \text{ in the dataset})$$

In Words: Simply the value (or values) that shows up more often than any other in the dataset.

ML Use Case: Used to describe the most common class in a classification dataset, and to detect class imbalance.

Worked Example: Data: 4, 8, 6, 8, 2. The value 8 appears twice (more than any other), so Mode = 8.

4.2 Descriptive Statistics – Measures of Spread**Range****Symbols / Details:**

- Range – simplest measure of spread

$$\text{Range} = x_{\max} - x_{\min}$$

In Words: The distance between the largest and smallest value in the dataset.

ML Use Case: A quick sanity check when normalizing/scaling features (e.g. Min-Max scaling uses this exact quantity).

Worked Example: Data: 4, 8, 6. Range = $8 - 4 = 4$.

Sample Variance

Symbols / Details:

- s^2 – sample variance (uses $n-1$, Bessel's correction, for an unbiased estimate)

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

In Words: Average the squared distance of each point from the mean, dividing by $n-1$ instead of n to correct for the bias of estimating the mean from the same sample.

ML Use Case: Reported alongside model performance metrics across cross-validation folds to show consistency, and used in feature scaling (StandardScaler).

Worked Example: Step by step – continuing data 4, 8, 6, $\bar{x} = 6$:

$$(4-6)^2 + (8-6)^2 + (6-6)^2 = 4 + 4 + 0 = 8$$

$$s^2 = \frac{8}{3-1} = \frac{8}{2} = 4$$

Sample Standard Deviation

Symbols / Details:

- s – sample standard deviation, in the same units as the data

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

In Words: The square root of the variance – brings the spread measure back into the same units as the original data, making it directly interpretable.

ML Use Case: Used to standardize features to zero mean / unit variance before training (a near-universal preprocessing step).

Worked Example: From $s^2 = 4$ above: $s = \sqrt{4} = 2$.

Z-score (Standardization)

Symbols / Details:

- z – how many standard deviations a point is from the mean

$$z = \frac{x - \bar{x}}{s}$$

In Words: Subtract the mean, then divide by the standard deviation – rescales any dataset to have mean 0 and standard deviation 1.

ML Use Case: The core operation of feature standardization (StandardScaler) used before training almost any distance-based or gradient-based ML model.

Worked Example: For data point $x = 8$ with $\bar{x} = 6$, $s = 2$: $z = \frac{8-6}{2} = 1$ – this point sits exactly 1 standard deviation above the mean.

Percentile / Quantile

Symbols / Details:

- Q_p – the value below which $p\%$ of the data falls

$$Q_p = x_{(\lceil p \cdot n / 100 \rceil)} \quad (\text{sorted data})$$

In Words: Sort the data, then find the value at the position corresponding to the desired percentage cutoff.

ML Use Case: Used to build box plots, detect outliers, and evaluate models with percentile-based metrics (e.g. p95 latency, p95 error).

Worked Example: Data sorted: 2, 4, 6, 8, 10 ($n = 5$). The 40th percentile: position = $\lceil 0.4 \times 5 \rceil = 2 \Rightarrow Q_{40} = 4$.

Interquartile Range (IQR)

Symbols / Details:

- Q_1, Q_3 – 25th and 75th percentile

$$\text{IQR} = Q_3 - Q_1$$

In Words: The spread of the “middle 50%” of the data, ignoring extreme values at either end.

ML Use Case: Standard method for outlier detection: points beyond $Q_1 - 1.5 \cdot \text{IQR}$ or $Q_3 + 1.5 \cdot \text{IQR}$ are flagged as outliers before model training.

Worked Example: If $Q_1 = 4$ and $Q_3 = 10$: $\text{IQR} = 10 - 4 = 6$. Outlier cutoffs: below $4 - 1.5(6) = -5$ or above $10 + 1.5(6) = 19$.

4.3 Shape of a Distribution

Skewness

Symbols / Details:

- γ_1 – skewness – measures asymmetry of the distribution

$$\gamma_1 = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^3}{\left(\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \right)^{3/2}}$$

In Words: Average the CUBED distances from the mean (which keeps the sign, unlike squaring) and normalize by the standard deviation cubed. Positive skew means a long tail stretching to the right; negative skew means a long tail to the left; zero means symmetric.

ML Use Case: Used to detect skewed target variables or features (e.g. income, price data) that

often benefit from a log-transform before model training.

Worked Example: House prices like $\{200k, 220k, 210k, 900k\}$ have a long right tail (one huge outlier pulling the mean up) $\Rightarrow \gamma_1 > 0$ (positive/right skew) – a signal to try a log-transform.

Kurtosis

Symbols / Details:

- Kurt – kurtosis – measures “tailedness” / how extreme the outliers of a distribution are

$$\text{Kurt} = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^4}{\left(\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2\right)^2}$$

In Words: Average the distances from the mean raised to the FOURTH power (which punishes extreme outliers heavily) and normalize by variance squared. A normal distribution has kurtosis 3.

ML Use Case: Used to detect heavy-tailed data (which can destabilize gradient-based training) and as a diagnostic for whether Gaussian-based assumptions in a model are reasonable.

Worked Example: A dataset with occasional extreme spikes (e.g. stock returns with rare crashes) will have $\text{Kurt} > 3$ – heavier tails than a normal distribution, meaning extreme values are more common than a Gaussian model would predict.

Excess Kurtosis

Symbols / Details:

- Excess Kurt – kurtosis relative to a normal distribution

$$\text{Excess Kurt} = \text{Kurt} - 3$$

In Words: Subtract 3 (the kurtosis of a perfect normal distribution) so that 0 means “normal-like tails,” positive means “heavier tails than normal” (more extreme outliers), and negative means “lighter tails.”

ML Use Case: A quick diagnostic used before choosing loss functions robust to outliers (e.g. Huber loss instead of MSE) when excess kurtosis is high.

Worked Example: If $\text{Kurt} = 5$ for a dataset, $\text{Excess Kurt} = 5 - 3 = 2$ – notably heavier-tailed than normal, suggesting outlier-robust methods.

4.4 Relationships Between Variables

Sample Covariance

Symbols / Details:

- s_{xy} – sample covariance between two variables x, y

$$s_{xy} = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

In Words: Average how much x and y move together relative to their means – positive if they tend to rise/fall together, negative if they move oppositely.

ML Use Case: The building block of the correlation coefficient below and of the covariance matrix used in PCA.

Worked Example: Study hours $x = (1, 2, 3)$, test scores $y = (50, 60, 70)$: $\bar{x} = 2, \bar{y} = 60$.

$$s_{xy} = \frac{(1-2)(50-60) + (2-2)(60-60) + (3-2)(70-60)}{3-1} = \frac{10+0+10}{2} = 10$$

Positive: more study hours go with higher scores.

Pearson Correlation Coefficient

Symbols / Details:

- r – correlation, always between -1 and 1

$$r = \frac{s_{xy}}{s_x s_y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

In Words: Rescale covariance by both variables' standard deviations so the result always falls between -1 (perfect negative linear relationship) and $+1$ (perfect positive linear relationship).

ML Use Case: Used in feature selection to drop highly correlated (redundant) features, and to detect multicollinearity before linear regression.

Worked Example: **Step by step:** same data, $x = (1, 2, 3)$, $y = (50, 60, 70)$.

$$\sum (x_i - \bar{x})^2 = 1 + 0 + 1 = 2, \quad \sum (y_i - \bar{y})^2 = 100 + 0 + 100 = 200$$

$$r = \frac{20}{\sqrt{2}\sqrt{200}} = \frac{20}{\sqrt{400}} = \frac{20}{20} = 1$$

A perfect positive linear relationship – every extra study hour adds exactly 10 points.

Coefficient of Determination (R^2)

Symbols / Details:

- SS_{res} – residual sum of squares
- SS_{tot} – total sum of squares

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

In Words: The fraction of the total variation in y that the model successfully explains; $R^2 = 1$ means perfect fit, $R^2 = 0$ means the model is no better than just predicting the mean.

ML Use Case: The standard headline metric reported for regression model performance.

Worked Example: If $SS_{res} = 20$ (leftover error after the model's predictions) and $SS_{tot} = 200$

(total variation in y around its mean):

$$R^2 = 1 - \frac{20}{200} = 1 - 0.1 = 0.9$$

The model explains 90% of the variance in y .

4.5 Estimation and Confidence

Standard Error of the Mean

Symbols / Details:

- SE – standard error – uncertainty in the sample mean as an estimate of μ

$$SE = \frac{s}{\sqrt{n}}$$

In Words: The standard deviation shrinks by a factor of $\sqrt{n}$ when you're talking about the mean of n samples instead of a single sample – more data means a more precise estimate of the true mean.

ML Use Case: Used to construct confidence intervals for model performance metrics and to judge whether a difference in accuracy across models is meaningful or just noise.

Worked Example: With $s = 2$ and $n = 4$ samples: $SE = \frac{2}{\sqrt{4}} = \frac{2}{2} = 1$.

Confidence Interval (for the mean)

Symbols / Details:

- z^* – critical value from the normal distribution (e.g. 1.96 for 95%)

$$CI = \bar{x} \pm z^* \cdot \frac{s}{\sqrt{n}}$$

In Words: A range of plausible values for the true population mean, centered on the sample mean, wide enough to be right about 95% (or whatever confidence level chosen) of the time.

ML Use Case: Used to report a range around a model's estimated accuracy/error rather than a single misleadingly-precise number.

Worked Example: **Step by step:** $\bar{x} = 6$ (our mean example), $s = 2$, $n = 3$, using $z^* = 1.96$ for 95%:

$$SE = \frac{2}{\sqrt{3}} \approx 1.155, \quad CI = 6 \pm 1.96(1.155) = 6 \pm 2.26$$

So the 95% confidence interval is roughly (3.74, 8.26).

Z-test Statistic

Symbols / Details:

- μ_0 – the hypothesized population mean under H_0

$$z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}}$$

In Words: Measures how many standard errors the observed sample mean is away from the value assumed under the null hypothesis.

ML Use Case: Used in A/B testing to decide whether a new model/feature meaningfully changed a metric, or the difference could just be random chance.

Worked Example: Test $H_0 : \mu_0 = 5$, observed $\bar{x} = 6$, $\sigma = 2$, $n = 4$:

$$z = \frac{6 - 5}{2/\sqrt{4}} = \frac{1}{1} = 1$$

A z -score of 1 is not extreme (typically need $|z| > 1.96$ for significance at $\alpha = 0.05$), so we would not reject H_0 .

p-value (conceptual definition)

Symbols / Details:

- **p-value** – probability of seeing data this extreme (or more) if H_0 were actually true

$$p = P(|\text{Test Statistic}| \geq |\text{observed}| \mid H_0 \text{ true})$$

In Words: If the null hypothesis were true, the p-value is the chance you'd still see a result at least this extreme just by random luck. A small p-value (typically $< \alpha = 0.05$) means the observed effect is unlikely to be pure chance.

ML Use Case: Used throughout A/B testing and statistical significance testing of model improvements before shipping changes.

Worked Example: From the z -test above, $z = 1$ corresponds to a two-tailed p -value of about 0.317. Since $0.317 > 0.05$, we do not have enough evidence to reject H_0 .

Chi-Square Statistic

Symbols / Details:

- O_i – observed frequency
- E_i – expected frequency under H_0

$$\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i}$$

In Words: Sum the squared, normalized gap between what you actually observed and what you'd expect if the null hypothesis were true.

ML Use Case: Used for feature selection with categorical variables (chi-square test of independence) and goodness-of-fit tests.

Worked Example: Rolling a die 60 times, expecting $E_i = 10$ per face; observed counts are

8, 12, 9, 11, 10, 10:

$$\chi^2 = \frac{(8-10)^2}{10} + \frac{(12-10)^2}{10} + \dots = \frac{4 + 4 + 1 + 1 + 0 + 0}{10} = 1.0$$

A small χ^2 suggests the die's rolls are close to what a fair die would produce.

4.6 Statistics in Model Fitting

Mean Squared Error (MSE)

Symbols / Details:

- y_i – true value
- $\hat{y}_i$ – predicted value

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

In Words: Average the squared gap between each prediction and its true value – big errors are punished disproportionately harder than small ones because of the square.

ML Use Case: The default loss function for regression problems, and mathematically the negative log-likelihood of a Gaussian noise model (connecting statistics back to probability).

Worked Example: True values $y = (3, 5, 2)$, predictions $\hat{y} = (2, 5, 4)$:

$$MSE = \frac{(3-2)^2 + (5-5)^2 + (2-4)^2}{3} = \frac{1 + 0 + 4}{3} = \frac{5}{3} \approx 1.67$$

Root Mean Squared Error (RMSE)

Symbols / Details:

- $RMSE$ – MSE brought back into the original units

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

In Words: Take the square root of MSE so the error is expressed in the same units as the original target variable, making it easier to interpret.

ML Use Case: The most commonly reported regression evaluation metric alongside R^2 .

Worked Example: From $MSE \approx 1.67$ above: $RMSE = \sqrt{1.67} \approx 1.29$ – on average, predictions are off by about 1.29 units.

Mean Absolute Error (MAE)

Symbols / Details:

- MAE – average absolute error

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

In Words: Average the absolute (unsquared) gap between predictions and truth – treats all error sizes proportionally, unlike MSE.

ML Use Case: Preferred over MSE/RMSE when the dataset has outliers you don't want to dominate the error metric.

Worked Example: Same data $y = (3, 5, 2)$, $\hat{y} = (2, 5, 4)$:

$$MAE = \frac{|3 - 2| + |5 - 5| + |2 - 4|}{3} = \frac{1 + 0 + 2}{3} = 1$$

Simple Linear Regression Model

Symbols / Details:

- β_0, β_1 – intercept and slope parameters
- ϵ – random noise / error term

$$y = \beta_0 + \beta_1 x + \epsilon$$

In Words: Model the target y as a straight-line function of x , plus some unavoidable random noise.

ML Use Case: The simplest possible regression model, and the conceptual starting point for all linear models including logistic regression and linear layers in neural networks.

Worked Example: From the study-hours example ($r = 1$, perfect fit): $y = 40 + 10x$ (intercept $\beta_0 = 40$, slope $\beta_1 = 10$). Predicts a score of $40 + 10(3) = 70$ for 3 study hours – matching the data exactly.

Ordinary Least Squares (Normal Equation)

Symbols / Details:

- X – design matrix of features
- $\mathbf{y}$ – vector of targets
- $\hat{\beta}$ – estimated coefficients

$$\hat{\beta} = (X^\top X)^{-1} X^\top \mathbf{y}$$

In Words: The closed-form formula that directly computes the coefficients minimizing the sum of squared errors, without needing gradient descent.

ML Use Case: The exact formula used by scikit-learn's `LinearRegression` under the hood for small-to-medium datasets.

Worked Example: For the study-hours data, plugging X (with a column of 1's for the intercept, plus study hours) and $\mathbf{y} = (50, 60, 70)$ into this formula yields exactly $\hat{\beta} = (40, 10)^\top$, matching $y = 40 + 10x$ from above.

Ridge Regression (L2 Regularization) Cost

Symbols / Details:

- λ – regularization strength

$$J(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

In Words: Minimize the usual squared error, but also penalize large coefficient values, shrinking them toward zero to reduce overfitting.

ML Use Case: The statistical origin of “weight decay,” used in virtually every modern deep learning optimizer to control model complexity.

Worked Example: If squared error = 5 and $\beta = (10, 2)$ with $\lambda = 0.1$: penalty = $0.1(100 + 4) = 10.4$, so $J = 5 + 10.4 = 15.4$ – a large coefficient like 10 gets heavily penalized, pushing the optimizer to prefer smaller weights.

Lasso Regression (L1 Regularization) Cost

Symbols / Details:

- λ – regularization strength

$$J(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

In Words: Same idea as Ridge, but penalizing the absolute value of coefficients instead of their square – this tends to push some coefficients to exactly zero, performing automatic feature selection.

ML Use Case: Used when you want a sparse, more interpretable model that automatically discards irrelevant features.

Worked Example: Same $\beta = (10, 2)$, $\lambda = 0.1$: penalty = $0.1(10 + 2) = 1.2$, so $J = 5 + 1.2 = 6.2$. Unlike Ridge, Lasso's penalty grows linearly, which can drive small, unhelpful coefficients all the way to exactly 0.

Bias–Variance Decomposition

Symbols / Details:

- Bias – systematic error from overly simple assumptions
- Var – sensitivity to fluctuations in the training data

$$E[(y - \hat{f}(x))^2] = \text{Bias}(\hat{f}(x))^2 + \text{Var}(\hat{f}(x)) + \sigma_\epsilon^2$$

In Words: Total expected error splits cleanly into three parts: how wrong the model is on average (bias), how much its predictions swing across different training sets (variance), and noise that no model can ever remove.

ML Use Case: The central theoretical framework for understanding overfitting vs. underfitting and for choosing model complexity / regularization strength.

Worked Example: A straight-line model fit to curvy data has high bias (systematically wrong shape) but low variance (stable across resamples). A very flexible model (e.g. a deep decision tree) can have low bias but high variance (wildly different fits on slightly different training sets) – the classic overfitting trade-off.